

ARD2: skyline full-context anchored row diagrams

DEFINITIONS & PROOFS / ENGLISH EDITION

2026-09-17. PDF · NER · Python · Well-ordering and exact equivalence · Lean.

ARD2 is the full-context version of Anchored Row Diagrams. The default edition now removes dominated records and replaces a whole lower-row package with one controller predecessor. Its standard domain is **order-isomorphic to ARD2-legacy**, with indexed fundamental-sequence commutation and unchanged local counts. The old programs, manuscripts and Lean sources are preserved.

1. Columns and skylines

A finite term is a column string $G = (C_0, \dots, C_{m-1})$. Entries at child j are

$$(k, p, q), \quad 0 \leq k, q \leq j, \quad 0 \leq p < j.$$

The coordinates are row anchor, parent and maximum root. The maximum denotes all roots $0, \dots, q$. Unlike ARD, row and root need not precede the parent and may both equal the child j , called **SELF**. SELF is that child's address and moves with it; parents remain strictly earlier.

An entry's priority is the lexicographically ordered pair (k, q) . The skyline $S(C)$ is obtained as follows:

1. Keep only the greatest priority at each parent.
2. Scan parents in decreasing order and keep only strict priority record highs.

Thus parents strictly decrease and priorities strictly increase. Empty columns are not discarded. Zero has no columns and is written $\emptyset$; one is $[]$; m empty columns denote the natural number m . A complete list example is

```
[ ][(1,0,1)][(2,1,2)]
```

Entries still have only three natural coordinates, without trees, expansion histories or additional parameters. Constructors check all original coordinates before taking a skyline, so dominance cannot hide an illegal coordinate. Old lists are accepted and compressed, without certifying standard reachability.

2. Column comparison

The entry comparison key is (p, k, q) . Compare the decreasing entry lists lexicographically, then compare graphs lexicographically from the leftmost column. Proper prefixes are smaller at both levels. Comparison runs neither expansion nor path search.

Adjoin a maximum **Top**, displayed as **Limit** of ARD2. It is not a finite column or a usable coordinate. The column order on **arbitrary legal graphs is not well-founded**; the well-order claim concerns the standard domain in Section 4.

3. Fundamental sequences

Let $n \in \mathbb{N}$. Set $0[n] = 0$, excluding this self-loop from strict steps. For nonzero finite graphs,

$$G[0] = G \text{ with its final column deleted.}$$

An empty final column is deleted for every index. Otherwise put $x = m - 1$ and let (K, c, R) be the final entry of C_x , the controller. Set

$$L = x - c > 0, \quad N_b = x + bL, \quad \phi_b(i) = \begin{cases} i, & i < c, \\ i + bL, & i \geq c. \end{cases}$$

Movement acts on all three coordinates. For an already moved controller define

$$\delta_N(k, p, q) = \begin{cases} \{(k, p, q - 1)\}, & q > 0, \\ \{(k - 1, p, N)\}, & q = 0 < k, \\ \emptyset, & k = q = 0. \end{cases}$$

The borrow ceiling is the **new seam child** N , not parent p . Move before taking the predecessor in every block; do not copy a predecessor computed only once.

Retain $C_0, \dots, C_{x-1}$. For $b = 0, \dots, n - 1$ append

$$J_b = S(\phi_b(C_x \setminus \{(K, c, R)\}) \cup \delta_{N_b}(\phi_b(K, c, R)) \cup \phi_{b+1}(C_c)),$$

followed by $\phi_{b+1}(C_{c+1}), \dots, \phi_{b+1}(C_{x-1})$. Every source is read from the frozen input graph, never from newly generated output.

The cut column C_c uses movement $b + 1$, with $\phi_{b+1}(c) = N_b$. Both its row SELF and root SELF therefore rebind to the seam child; its parents remain below the original cut. In contrast to ARD, one cannot simply merge an unmoved C_c .

For $H = A_2$:

```
H      = [][(1,0,1)]
H[0]   = []
H[1]   = [][(1,0,0)]
H[2]   = [][(1,0,0)][(2,1,1)]
H[1][1] = [][(0,0,1)]
```

The output has $x + nL$ columns. Every column before the old final one is unchanged, and $G[n]$ is a full-column prefix of $G[n + 1]$, strict when the old last column is nonempty. This requirement does not apply to the external top.

4. Seeds, standardness and proof scope

The seed A_n has n columns, with $C_0 = \emptyset$ and $C_j = \{(j, j - 1, j)\}$ for $j > 0$; set $\text{Top}[n] = A_n$. The finite standard domain consists of their finite expansion descendants. Accessibility, semantic representability and well-foundedness are not built into standardness.

The paper proves:

- Nonzero expansion is well-founded on all legal skyline graphs.
- The specified column order on standard graphs is a well-order, also after adjoining the top.
- Columnwise compression Q satisfies $Q(E_n G) = F_n(QG)$ for the old and new rules E, F . It is a standard-domain order isomorphism and preserves local counts. It need not be injective on arbitrary raw legal graphs.

The paper upper bound remains KP_ω + “there exists an uncountable ordinal”, with full set induction and no added power set or choice. The new Lean entry proves the new rule well-ordered by explicitly reusing the old semantic backend. **The exact isomorphism, internal weak-object-theory derivation and cross-notation comparisons are not Lean-formalized.** This is not Python/JS virtual-machine verification.

The paper embedding ARD below ARD2(1,3) transfers through the isomorphism. Its fixed target $[][(0,0,1)]$ is already a skyline. This revision establishes no whole-system comparison with wY, CWY2 or IPD.

5. Local counts and five displays

Freeze a column's prefix, treat the column as final, expand at a positive index, and discard newly appended columns. Repeat until that position is deleted. The local count includes the final empty-column deletion, so an empty column has count one. Compression preserves width, truncation and nonemptiness and commutes with this local step, so counts agree with the legacy edition. Seed counts are prefixes of

```
1, 5, 55, 969, 23751, ...
```

The counter uses exact integers; its work budget is not a mathematical rule. There are finitely many legal skyline columns at a fixed position and each local step strictly decreases their order, so every local count terminates.

NER retains five displays: default **list**, plus **count sequence**, **arc diagram**, **text adjacency table**, **drawn adjacency tables**. There is no duplicate list menu item. Text adjacency uses [] per column, semicolons for row anchors, commas for parents and blank cells for absence; zero is not absence. For example A_2 becomes [] [; 1]. Drawn adjacency uses one complete compact upper triangle per row anchor, light diagonal cells as indices, and the count sequence above all tables. Both diagrams display all current skyline entries, not hidden legacy redundancy.

Fonts inherit the page. Counting, geometry and expansion have isolated budgets, retaining roughly one second plus width, work and canvas limits. Exhaustion is explicit: no approximate counts and no partial diagram presented as complete.

6. Use and checks

Import the standalone JS into **NER** and choose **ARD2**. Its new ID is ard2-skyline-v02; the old edition independently registers ard2-legacy-v01 with display name **ARD2-legacy**, allowing simultaneous import. Previously loaded browser code does not update automatically; reimport the file.

Inputs include A_3 , $A_3[2][1]$, $\text{Limit}[3]$, Limit of **ARD2**, natural numbers and complete lists. A natural number counts empty columns; it is not a count-word encoding. FS , FS_alter and FS_short are identical, with no index shift.

```
from ard2 import ARD2, TOP
a = ARD2.seed(3)
assert TOP[3] == a
assert a[1] < a
assert a[1].columns == a[2].columns[:len(a[1].columns)]
assert a.local_step().columns == a[1].columns[:len(a.columns)]
```

Python uses arbitrary-precision integers and intentionally omits graphics, caching and resource guards. Bound experiments in the caller. From the repository root:

```
python -B tests/test_ard2.py
python -B tests/test_ard2_legacy.py
node --max-old-space-size=256 tests/ard2_ner.cjs
node --max-old-space-size=256 tests/ard2_display.cjs
```

Tests cover an independent full-root oracle, exact legacy commutation, both SELF coordinates, zero indices, prefixes, comparisons, local counts and complete displays. Finite tests do not prove the isomorphism or well-ordering.